Linux Server Compromise

Incident response and forensics checklist · work through the phases in order · times in UTC, every action written down

The golden rules. (1) **Do not reboot or power off** - memory holds the running malware, its network connections and often its decryption keys, and it is gone at power-off. (2) **Do not "clean up"** - no deleting files, killing processes, or running antivirus on the live box before evidence is collected. (3) **Assume the attacker is watching** - talk out-of-band (phone, Signal), not over email or chat that runs through the compromised systems. (4) **Collect most volatile first** (RFC 3227): memory → network → processes → disk → logs. (5) **Never write evidence to the suspect disk** - send it to external or remote storage and hash every file.

Phase overview

#	Phase	Goal
0	Pause and organize	Stop yourself from destroying evidence; set up notes and people
1	Contain the network	Cut the attacker off without powering the box down
2	Volatile evidence	Memory image, live network and process state
3	Disk and logs	Bit-for-bit disk image, all logs copied off the server
4	Analyze (on copies)	Timeline, entry vector, persistence, scope, data loss
5	Eradicate and recover	Rebuild clean, rotate every secret, patch the hole
6	Post-incident	Notifications, report, lessons learned, better logging

Phase 0 - Pause and organize (first 15 minutes)

- ☐ **Stop.** Do not log in again and "look around" with the system's own tools more than you must; every command changes timestamps and memory.
- ☐ Start an **incident log** (paper or a machine you trust): UTC time, who, what command, what result. This becomes your timeline and your chain-of-custody record.
- ☐ Record how the compromise was noticed: alert, GuardDuty finding, odd process, customer report, ransom note. Screenshot or photograph it.
- ☐ Decide who leads. Inform management. If legal action, insurance, or personal data may be involved, bring in **legal counsel now** - and consider a professional DFIR firm before you touch anything.
- ☐ Check notification deadlines that may now be running (e.g. GDPR: 72 hours to the regulator after becoming aware of a personal-data breach).
- ☐ Work from a **clean workstation**. Do not use credentials or SSH keys that live on the compromised server to reach other systems.
- ☐ Prepare evidence storage: an external drive, a separate S3 bucket (ideally Object Lock), or a forensics host reachable over the network - with enough space for RAM + disk images.

Phase 1 - Contain the network (keep the machine running)

Goal: stop exfiltration, lateral movement and remote control, while the system keeps running so memory survives.

On AWS EC2

- ☐ Enable **termination protection** and add tags (incident ID, do-not-touch).

- ☐ Detach the instance from its **Auto Scaling group** (otherwise it may be terminated and replaced) and deregister it from **load balancers**.
- ☐ Swap its security groups for an **isolation security group** that allows only your forensics IP. Note: security-group changes do not cut connections that are already tracked - add a **network ACL deny** on the subnet (if other hosts allow it) to kill existing sessions immediately.
- ☐ **Snapshot every EBS volume now** (cheap, fast, and protects you if the box is lost).
- ☐ Revoke the instance role's active sessions (IAM → role → *Revoke active sessions*) - stolen instance credentials keep working otherwise.
- ☐ Pull **GuardDuty findings, CloudTrail, VPC Flow Logs**, LB and Route 53 Resolver logs for the period.

On-premises / other clouds

- ☐ Move the machine to an isolated VLAN, or block it at the upstream firewall/switch port. Do **not** pull the power cable; unplugging the network cable is acceptable if nothing else is possible.
- ☐ If the hypervisor allows it, take a **VM snapshot including memory** (VMware .vmsn/.vmem, KVM `virsh dump --memory-only`) - this is often the best memory image you can get.

> **If the attack is actively destroying data** (ransomware encrypting, wiper running), containment beats evidence: isolate immediately, and if the damage continues, power off. Lost memory is better than lost data.

Phase 2 - Collect volatile evidence

Use **trusted, statically linked tools** from read-only media or a mounted volume - the system's own `ps`, `ss` and `ls` may be trojaned by a rootkit. Write all output to external storage (`/mnt/evidence` below) or stream it over the network, never to the suspect disk.

2a. Memory image (first!)

```
# AVML (Microsoft, single static binary, no kernel module needed)
sudo ./avml /mnt/evidence/mem.lime
# alternative: LiME kernel module built for this exact kernel
sudo insmod lime-$(uname -r).ko "path=/mnt/evidence/mem.lime format=lime"
sha256sum /mnt/evidence/mem.lime > /mnt/evidence/mem.lime.sha256
```

Analyze later with **Volatility 3** (needs a symbol table for the kernel - record `uname -a` and the distro kernel package version).

2b. Live system state

```
E=/mnt/evidence/live; mkdir -p $E
date -u > $E/date.txt;  uptime > $E/uptime.txt;  uname -a > $E/uname.txt
w > $E/w.txt;  who -a > $E/who.txt;  last -Faiwx > $E/last.txt;  lastb -Faiw > $E/lastb.txt
# network
ss -tunapoe > $E/ss.txt;  ip -d addr > $E/ip_addr.txt;  ip route > $E/ip_route.txt
ip neigh > $E/arp.txt;  conntrack -L > $E/conntrack.txt 2>&1
nft list ruleset > $E/nft.txt 2>&1;  iptables-save > $E/iptables.txt 2>&1
cat /etc/resolv.conf /etc/hosts > $E/dns_hosts.txt
# processes
ps auxwwf > $E/ps.txt;  ps -eo pid,ppid,user,lstart,cmd > $E/ps_start.txt
lsof -nP > $E/lsof.txt 2>&1
ls -la /proc/*/exe 2>/dev/null > $E/proc_exe.txt      # look for "(deleted)" binaries
for p in /proc/[0-9]*; do echo "$p $(tr '\0' ' ' < $p/cmdline)"; done > $E/cmdlines.txt
# kernel, mounts, persistence hints
lsmod > $E/lsmod.txt;  cat /proc/modules > $E/proc_modules.txt;  mount > $E/mount.txt
cat /etc/ld.so.preload > $E/ld_so_preload.txt 2>&1     # should not exist / be empty
systemctl list-units --all > $E/units.txt;  systemctl list-timers --all > $E/timers.txt
for u in $(cut -d: -f1 /etc/passwd); do
  echo "== $u"; crontab -l -u $u 2>/dev/null
done > $E/crontabs.txt
cd $E && sha256sum * > SHA256SUMS
```

2c. Suspicious processes - copy, do not kill

```
cp /proc/<PID>/exe /mnt/evidence/proc_<PID>.bin       # recovers even deleted binaries
cat /proc/<PID>/maps /proc/<PID>/environ > /mnt/evidence/proc_<PID>_info.txt
ls -la /proc/<PID>/fd /proc/<PID>/cwd > /mnt/evidence/proc_<PID>_fd.txt
```

2d. Network capture

- ☐ Start a packet capture from **outside** the box if possible (AWS Traffic Mirroring, switch SPAN port); otherwise `tcpdump -i any -w /mnt/evidence/cap.pcap`.
- ☐ If you run TLS key logging (eCapture / SSLKEYLOGFILE), secure a copy of the key log with the capture.
- ☐ Note every remote IP / domain in `ss` and `conntrack` output - these are your first indicators of compromise (IOCs), e.g. C2 servers.

Phase 3 - Disk image and logs

3a. Disk image

- ☐ **AWS:** EBS snapshots (Phase 1) are your images. Create volumes from them and attach them to a separate **forensics instance** in an isolated VPC/account - never back to the original.
- ☐ **Physical/VM:** image with `dc3dd`, `ewfacquire` or `dd` to external storage, ideally from a forensic live USB with a write blocker.
- ☐ Hash the image (`sha256sum`) at acquisition and again before every analysis session.
- ☐ Only now, if required, shut the machine down - or keep it isolated for later inspection.

```
sudo dc3dd if=/dev/nvme0n1 of=/mnt/evidence/disk.img hash=sha256 log=/mnt/evidence/disk.log
```

3b. Logs - copy everything, attacker may wipe local copies

Source	Where / how
System journal	`journalctl --output=export > journal.export`, plus `/var/log/journal/`
Auth / logins	`/var/log/auth.log` or `/var/log/secure`, `/var/log/wtmp`, `btmp`, `lastlog`
Audit	`/var/log/audit/` (auditd), eBPF/Falco/osquery output if installed
Shell history	`~/.bash_history`, `~/.zsh_history`, `~/.python_history`, `~/.mysql_history` for **every** user incl. root
Applications	Web server access/error logs, database logs, app logs, mail logs, package manager logs (`/var/log/apt/`, `dnf.log`)
Cloud	CloudTrail, VPC Flow Logs, GuardDuty, ALB/NLB, WAF, Route 53 Resolver query logs, S3 access logs
Central	SIEM / remote syslog copies - most trustworthy, since the attacker could not edit them

Record the **chain of custody** for every evidence item: what it is, hash, who collected it, when, where it is stored, and every hand-over.

Phase 4 - Analyze (on copies only)

Mount images **read-only**: `mount -o ro,noexec,nodev,noload /dev/xvdf1 /mnt/case` (`noload` skips ext4 journal replay). Never run binaries from the image.

4a. Build a timeline

- ☐ Super-timeline with **plaso / log2timeline**, or quickly: `find /mnt/case -newermt '2026-10-01' ! -newermt '2026-10-23' -ls`.
- ☐ Correlate: first suspicious log line, file creation times, logins, outbound connections in flow logs.

- ☐ Remember attackers alter mtimes (timestomping) - ctime and ext4 crtime are harder to fake; use `debugfs -R 'stat <inode>'` or `istat` (Sleuth Kit).

4b. Find the initial entry vector

- ☐ SSH: successful logins from unknown IPs, brute-force followed by success, new keys.
- ☐ Web: exploit requests in access logs (long/odd URLs, POSTs to upload paths, 500 bursts), web shells in the docroot.
- ☐ Unpatched services: compare installed versions against known CVEs (exposed admin panels, CI, databases).
- ☐ Stolen credentials: leaked API keys / cloud keys (check CloudTrail for use from unusual IPs).
- ☐ Supply chain: recently installed or updated packages, containers, scripts piped from curl.

4c. Hunt for persistence

Location	What to look for
Accounts	New users, UID 0 duplicates in `/etc/passwd`, changed `/etc/shadow`, `/etc/sudoers` and `sudoers.d/`
SSH	`~/.ssh/authorized_keys` of every user, `sshd_config` changes
Scheduled tasks	`/etc/cron*`, `/var/spool/cron/`, systemd timers, `at` jobs
Services	New/changed units in `/etc/systemd/system/`, `~/.config/systemd/user/`, `/etc/rc.local`, init.d
Shell startup	`.bashrc`, `.profile`, `/etc/profile.d/`, `/etc/bash.bashrc`
Library hijack	`/etc/ld.so.preload`, `LD_PRELOAD` in units, new files in `/etc/ld.so.conf.d/`
Auth stack	Modified or extra PAM modules (`/etc/pam.d/`, `/lib/*/security/`) - a classic password stealer / backdoor
Kernel	Unknown kernel modules, eBPF programs (`bpftool prog list` on the live box)
Binaries	Packages that fail verification: `debsums -c` / `rpm -Va`; new SUID files (`find / -perm -4000`)
Hidden files	`/tmp`, `/var/tmp`, `/dev/shm`, dot-directories, names like `"..."` or `" "`
Containers / cloud	Unknown images or containers, new IAM users/keys/roles, Lambda functions, EC2 instances in unused regions

4d. Scan and scope

- ☐ Scan the image with **YARA** rules, **ClamAV**, **rkhunter**/**chkrootkit** (run against the mounted image, not the live box). Upload hashes - not files with your data - to VirusTotal.
- ☐ Check IOCs (IPs, domains, hashes) against threat intel (abuse.ch ThreatFox, MISP, GuardDuty) and identify the malware family / C2 framework.
- ☐ **Lateral movement:** did the attacker use this server's keys, passwords or IAM role to reach other systems? Search the same IOCs across all hosts and cloud accounts.
- ☐ **Data exfiltration:** large outbound transfers in flow logs, archives created (tar, zip, 7z), database dumps, access to secrets files. This decides your notification obligations.

Phase 5 - Eradicate and recover

> **Rebuild, do not clean.** Once root was compromised you cannot prove the system is clean - rootkits, modified binaries and firmware-level persistence survive "cleanup". Reinstall from known-good media / AMI.

- ☐ Build a **new** server from a trusted image and configuration management - not from the compromised disk.
- ☐ **Fix the entry vector first** (patch, close the port, remove the vulnerable app), or the attacker walks back in.
- ☐ **Rotate every secret** that was on or used by the server - from a clean machine: SSH host and user keys, passwords, API tokens, database credentials, cloud access keys, IAM roles, TLS private keys (revoke and reissue certs), OAuth/session secrets, VPN credentials, keys in `.env` files and CI.
- ☐ Restore data only from backups taken **before** the earliest compromise time found in Phase 4, and verify them.
- ☐ Re-scan restored data/code for web shells or planted changes (diff against git).
- ☐ Block the known IOCs (C2 IPs/domains) at the firewall, DNS and in GuardDuty threat lists.
- ☐ Watch the new server and the rest of the fleet closely for weeks - attackers often return.

Phase 6 - Post-incident

- ☐ Notifications with legal counsel: regulators, affected customers/users, partners, cyber insurer, and law enforcement if appropriate.
- ☐ Write the incident report: timeline, entry vector, what was accessed, impact, actions taken, evidence list with hashes.
- ☐ Lessons-learned meeting within two weeks: what detected it, what delayed you, what was missing.
- ☐ Retain evidence according to legal advice (often years); keep it encrypted with restricted access.

Before the next incident - preparation checklist

Most of the evidence above only exists if it was set up in advance:

- ☐ **Central, append-only logging** off the server (remote syslog/SIEM, CloudWatch, S3 with Object Lock).
- ☐ **VPC Flow Logs** to S3, **GuardDuty**, **CloudTrail** in all regions, Route 53 Resolver query logs.
- ☐ **auditd** rules for execve and connect, or an eBPF agent (Falco, Tetragon, osquery).
- ☐ Time sync (NTP/chrony) everywhere - correlation is impossible without it.
- ☐ A ready **forensics kit**: static busybox, AVML, LiME built for your kernels, dc3dd, an evidence drive, this checklist printed.
- ☐ Tested, offline/immutable **backups** with known retention.
- ☐ A contact list: incident lead, management, legal, DFIR firm, AWS support, insurer - stored off the servers.
- ☐ Rehearse once: run this checklist on a test VM.

References: NIST SP 800-61 (Incident Handling), NIST SP 800-86 (Forensic Techniques), RFC 3227 (Evidence Collection and Archiving), AWS Security Incident Response Guide.